Numerical Analysis (089180)
Prof. Carlo de Falco
carlo.defalco@polimi.it

A.Y. 2024-2025
Giuseppe Caliò
giuseppe.calio@polimi.it

Lab 11 – Solutions

December 16, 2024

This lab deals with numerical differentiation and solution of ODEs.

1 One step methods for ODEs

We consider Cauchy problems of the type:

find $y \in \mathcal{C}^1(I)$ such that

$$\begin{cases} y'(t) = f(t, y(t)) & t \in I \\ y(t_0) = y_0. \end{cases}$$

Forward Euler method

$$y_{n+1} = y_n + hf_n$$

Backward Euler method

$$y_{n+1} = y_n + hf_{n+1}$$

Crank-Nicolson method

$$y_{n+1} = y_n + \frac{h}{2}\left[f_n + f_{n+1}\right]$$

θ-method

$$y_{n+1} = y_n + h\left[\theta f_n + (1-\theta) f_{n+1}\right]$$

A method is **explicit** if y_{n+1} is computed in terms of previous values y_k, $k \leq n$. If y_{n+1} depends implicitly on itself through f the method is **implicit**.

Unitary Local Truncation Error
The unitary local truncation error is defined as

$$\tau_{n+1}^h = \frac{y(t_{n+1}) - u_{n+1}^*}{h}.$$

Order
A method is said of order p if

$$\tau_{n+1}^h = \mathcal{O}(h^p).$$

Consistency
A method is consistent if

$$\lim_{h \to 0} \tau_{n+1}^h = 0.$$

Convergence
A method is convergent if

$$\lim_{\substack{h\to 0,\\ nh=t}} y_n = y(t) \quad \forall n.$$

Euler methods are of first order. Crank-Nicolson method is of second order.

Euler methods

$$\tau_{n+1}^{(h)} = \frac{h}{2} y''(\xi) \quad \xi \in (t_n, t_{n+1})$$

Crank-Nicolson method

$$\tau_{n+1}^{(h)} = \frac{h^2}{6} y'''(\xi) \quad \xi \in (t_n, t_{n+1})$$

Exercise 11.1. Consider the following Cauchy problem

$$\begin{cases} y' = t - y + 1 & 0 \le t \le 1 \\ y(0) = 1 \end{cases}$$

whose solution is $y(t) = t + e^{-t}$.

a. Implement the Forward Euler method in a suitable function.

b. Determine the step size h needed by the Forward Euler method to approximate the solution in $T = 1$ with an absolute error smaller than 0.01. Numerically verify the result.

Solution Exercise 11.1.

ex_11_1.m

```
function [t,y] = forward_euler(f, y0, t0, T, h)
  %FORWARD_EULER Solves the ordinary differential equation y' = f(t, y), with
  %  y(0) = y0, in the nodes defined in t using Forward Euler method.
  %  y = FORWARD_EULER(f, y0, t0, T, h)
  %
  %  Inputs : f = function handle to the right hand side
  %           y0 = initial condition
  %           t  = nodes
  %  Outputs : y = approximated solution in the nodes

  time_istants=floor((T-t0)/h);
  t=linspace(t0, T, time_istants+1);
  y(:, 1) = y0;

  for n = 2:length(t)
    y(:, n) = y(:, n-1) + h* f(t(n-1), y(:, n-1));
  end

end
%% Exercise 11.1
% Implement the Forward Euler method in a suitable function.

% Determine the step size h needed by the Forward Euler method to
% approximate the solution in T = 1 with an absolute error
% smaller than 0.01. Numerically verify the result.
f   = @(t,y) t - y + 1;
y_ex = @(t) t + exp(-t);
%{
The following relation is employed:

abs(e)<= ( (M h) / (2 L) ) * exp(L(T-t0)) - 1);
where
h is the stepsize;
L is the Lipschitz constant of f(t, y), which is equal to 1 in our case;
```

```
I = [0, T], with T=1 in our case;
M = \max_{[0,T]} \abs{y''(t)}. In our case M = 1.
%}

tol = 0.01;
h = 2*tol/(exp(1) - 1);

t = [0:h:1+h]; % 1 is missing, since 1 si not a multiple of h.
%t = [t, 1];  % Add 1 to the list of nodess


[th, y_fe] = forward_euler(f, 1, 0, t(end), h);

err_fe_time = (abs(y_fe - y_ex(th)));
err_fe=max(err_fe_time);

figure
subplot(1,2,1)
hold on;
ezplot(y_ex, [0,1]);
plot(t, y_fe, 'r-');
legend('exact', 'FE');

subplot(1,2,2)
hold on;
plot(t, err_fe_time, 'r-');
legend('FE error');
```

Exercise 11.2. Consider the following Cauchy problem

$$\begin{cases} y' = -ty^2 & 0 \le t \le 2 \\ y(0) = 1 \end{cases}$$

whose solution is $y(t) = \dfrac{2}{t^2+2}$.

a. Implement the Backward Euler method in a suitable function, solving the nonlinear equation for y_{n+1}.

b. Determine the step size h such that the unitary local truncation error after the first step is smaller than 10^{-3}. Solve the problem using such step.

Solution Exercise 11.2.

ex_11_2.m

```
function [t, y, iter] = backward_euler(f, y0, t0, T, h)
  %BACKWARD_EULER Solves the ordinary differential equation y' = f(t, y), with
  %  y(0) = y0, in the nodes defined in t using Backward Euler method.
  %  y = BACKWARD_EULER(f, y0, t0, T, h)
  %
  %  Inputs : f = function handle to the right hand side
  %           y0 = initial condition
  %           t  = nodes
  %  Outputs : y = approximated solution in the nodes

  time_istants=floor((T-t0)/h);
  t=linspace(t0, T, time_istants+1);
  y(:, 1) = y0;
  iter=[];
  for n = 2:length(t)
    [y(:, n), ~,~, output] = fsolve(@(z) -z + y(:, n-1) + h*f(t(n), z), ...
       y(:, n-1), optimset('Display', 'off', 'MaxIter', 1000) );
    iter=[iter output.iterations];
  end

end
```

```
%% Exercise 11.2
clc; close all; clear all;
% Implement the Backward Euler method in a suitable function, solving the nonliner
% equation with newton method.


% Determine the step size h such that the unitary local truncation error after the
% first step is smaller than 10^-3. Solve the problem using such step.
% The LTE for Euler methods is
% tau(h) = h/2 * max(y''(t)) with t \in (tn, t(n+1))

f = @(t, y) - t*y.^2;

y_ex = @(t) 2 ./ (t.^2 + 2);
d2y_ex_dt = @(t) 4*(3*t.^2 -2) ./ (t.^2 + 2).^3;
fplot(d2y_ex_dt, [0, 2]);

% The plot shows that the maximum of y'' is attained at x = 0 and has absolute value 1.


tol = 1e-3;
h = 2*tol/1;
t0=0; T=2;
[t, y_be] = backward_euler(f, 1, t0, T, h);
% The unitary truncation error after the first step is
1/h * abs(y_ex(h) - y_be(2))
% The request is satisfied.
figure
fplot(y_ex, [0, 2]);
hold on;
plot(t, y_be, 'r-');
legend('exact', 'BE')
```

Exercise 11.3 (Exam July 20, 2023). Consider the following Cauchy problem

$$\begin{cases} y' = -\frac{1}{t}(2y + t^2 + y^2) & t \in [1, 5] \\ y(0) = 1 \end{cases}$$

Exam July 20, 2023

a. Let us approximate the given Cauchy problem using the Matlab function $forward\_euler.m$ with $h = 0.2$. Report the Matlab commands used and the values of the computed numerical solution.

b. Given that the exact solution is $y_{\text{es}}(t) = t^2(1 + \log(t))$, represent on the same graph the exact solution, the numerical solution computed in point a, and the numerical solution computed using the forward Euler method with $h = 0.1$. Upload the resulting graph in .png format.

c. Report the Matlab commands used and comment on the results obtained in point b.

d. Compute the numerical solution u_n for $n = 0, \dots, N$ of the given Cauchy problem with the forward Euler method and $h = [0.1,\ 0.05\ 0.025\ 0.0125]$. Compute the error $e_h = \max_{0 \le n \le N} |y(t_n) - y_{ex}|$, with $t_n = 1 + nh \quad n = 0 \dots N$ and $N = 4/h$. Report the error in a loglog plot.

e. Report the Matlab commands used in the computation of e_h and the values of e_h. Comment on the results obtained in point d in light of the theory.

Solution Exercise 11.3.

ex_11_3.m

```
%% Exercise 11.3
clc; clear; close all;
%% Point a)
f=@(t,y) -1./t.*(2*y+t.^2.*y.^2);
t0=1; y0=1; T=5; h=0.2;
```

```
[th, U] = forward_euler(f, y0, t0, T, h);
U
%% Point b)
y_ex= @(t) 1./(t.^2.*(1+log(t)));
[th2, U2] = forward_euler(f, y0, t0, T, 0.1);
time_vec=linspace(t0, T, 1000);
figure
plot(th, U, '-ro')
hold on
plot(th2, U2, '-b^')
plot(time_vec, y_ex(time_vec), 'g')
legend("h=0.2", "h=0.1", "Exact")
name=sprintf("comparison");
print(gcf,name, '-dpng');
% The numerical solution is more accurate the lower the steplenght.

%%
t0=1; err=[];
hvec=[0.1, 0.05,0.025,0.0125];
for h=hvec
   T=4/h;
   [tt, U] = forward_euler(f, y0, t0, T, h);
   err=[err max(abs(y_ex(tt)-U))];
end

figure
loglog(hvec, err,'-o')
hold on
loglog(hvec, hvec,'--')
grid on
legend("numerical err", "estimate", 'Location','best')
print(gcf, "error", '-dpng')
%%
err
% The error decreases linearly with h, in accordance with the theoretical
% results. The forward Euler method is, indeed, first order method.
```

Exercise 11.4 (Exam July 11, 2024). Consider the following Cauchy problem

$$\begin{cases} y' = e^{-t}\sin(y(t)) & 0 \le t \le 1 \\ y(0) = \pi/2 \end{cases}$$

Exam July 11, 2024

a. Let us approximate the given Cauchy problem using the Matlab function *backward_euler.m* with $h = 0.1$ and $h = 1$. Report the Matlab commands used.

b. Upload the plot of the solutions obtained in point a in a unique graph in .png format. Use hold on command and legend to distinguish the solutions. (Hint: use help to plot the curve with different colors and markers).

c. Comment the obtained results in point b regarding absolute stability, accuracy and computatinal complexity.

d. Solve the same problem of point a with the forward Euler method and $h = 2$. Report the Matlab commands used and the values of the solution at $t = 2$ and commment the results.

Solution Exercise 11.4.

ex_11_4.m

```
%% Exercise 11.4
clc; clear; close all;
%% Point a)
f= @(t,y) exp(-t)*sin(y);
t0= 0; T=10; y0=pi/2; h=0.1;
tic
[th1, U1, iter1]=backward_euler(f, y0, t0,T,h);
toc
h=1;
tic
[th2, U2, iter2]=backward_euler(f, y0, t0,T,h);
toc
%% Point b)
figure
plot(th1, U1, '-o')
hold on
plot(th2, U2, '-^')
legend("h=0.1", "h=1")
%% Point c)
% Regarding stability, it is observed that both numerical solutions are free
% from oscillations and are therefore stable.
% This aligns with the theory, which states that the backward Euler method
% is unconditionally absolutely stable.
%
% As for accuracy, we note significant differences between the two solutions.
% This is consistent with the fact that the solution for "large" values of h
% is not very accurate (although it is stable), and only with a "small" value
% of h is an accurate solution achieved.
%
% Regarding computational time, we can measure the time elapsed in both cases.
% We can conclude that for increasing h, the total computational time decreases
% (at the expense, as mentioned, of accuracy).

%% Point d)
h=2;
[t,y] = forward_euler(f, y0, t0, T, h);
y(2)
% The required value is obtained using the command y(2), as the first component
% of the vector contains the initial data for t = 0, and thus the second
% component contains the approximation for t = h = 2.
% The result is y(2) = 3.5708. Observing the behavior of the numerical solution,
% an oscillatory trend is evident, with y(2) being a maximum. This is
% 0consistent with the theory, which states that the forward Euler method is
% conditionally absolutely stable, specifically requiring
% h < 2/\lambda, where lambda is a value that depends on the function f.
```

Exercise 11.5 (Exam June 17, 2024). Consider the following Cauchy problem

$$\begin{cases} y' = -\frac{t+1}{t+2} y(t) & 0 \le t \le 1 \\ y(0) = 2 \end{cases}$$

The exact solution is $y_{\text{ex}}(t) = (t+2)e^{-t}$.

Exam Sep 6, 2024

a. Use the Matlab function *crank_nicolson.m* to approximate problem 1 using the Crank Nicolson method with step size $h = 0.5$. Note that the function *crank_nicolson.m* internally uses the Newton method. Report the approximations of $y(1)$ and $y(5)$, along with the Matlab commands used. Finally, provide the number of iterations performed by the Newton method to compute $y(1)$ and $y(5)$.

b. Use Matlab to graphically represent the exact solution $y_{\text{ex}}(t)$ over the interval $[0, 5]$, and overlay it on the graph of the numerical solution. Represent the exact solution with a solid line and the numerical solution with a dashed line. Upload the resulting graph.

c. Use the Crank Nicolson method to solve the problem with step size $h_1 = 1$, $h_2 = 0.5$,$h_1 = 0.25$,$h_1 = 0.125$ and compute the value of the errors:

$$e_h = \max_{0 \le n \le N} |y(t_n) - y_{ex}(t_n)|$$

Report the values of e_h and the Matlab commands used.

d. Use the obtained results to estimate the converge orfer of the method with respect to h. Comment on the results obtained in point d in light of the theory.

Solution Exercise 11.5.

ex_11_5.m

```
%% Exercise 11.5
clc; clear; close all;
%% Point a)
f= @(t, y) -(t+1)./(t+2).*y;
y_ex=@(t) (t+2).*exp(-t);
t0=0; T=5; h=0.5; y0=2;
[t, y, iter] = crank_nicolson(f, y0, t0, T, h);
y(1)
iter(1)
y(5)
iter(5)
%% Point b)
tvec=linspace(t0, T, 1000);
figure
plot(t, y, '--')
hold on
plot(tvec, y_ex(tvec))
legend("h=0.5", "exact")
name="numVSex";
print(gcf, name, '-dpng');
%% Point c)
hvec=[1,0.5,0.25,0.125]; err=[];
for h=hvec
   [t, y, iter] = crank_nicolson(f, y0, t0, T, h);
   err=[err max(abs(y-y_ex(t)))];
end
err
%% Point d)
p=log(err(1:end-1)./err(2:end))/log(2)
% The Crank Nicolson Method shows second order convergence as predicted by
% the theory.
figure
loglog(hvec, err)
hold on
loglog(hvec, hvec.^2)
legend("err", "O(h^2)", 'Location','south')
```

Exercise 11.6. Consider the Cauchy problem

$$\begin{cases} y' = -2xy & 0 \le x \le 1 \\ y(0) = 1 \end{cases}$$

with exact solution $y(x) = e^{-x^2}$.

a. Starting from the function for the Backward Euler method, implement the θ-method.

b. Consider the cases $\theta = 0$ (BE), 1/4, 1/2 (crank nicolson), 3/4 and 1 (FE). Apply the method with $h = 2^{-n}$ for $n = 4, \dots, 8$ and approximate the convergence order.

Solution Exercise 11.6.

ex_11_1.m

```
function [t, y, iter] = theta_method(f, y0, t0, T, h, theta)
  %THETA_METHOD Solves the ordinary differential equation y' = f(t, y) with
  %  y(0) = y0 in the nodes defined in t using the theta-method.
  %  y = THETA_METHOD(f, y0, t0, T, h, theta)
  %
  %  Inputs : f  = function handle to the right hand side
  %           y0  = initial condition
  %           t   = nodes
  %           theta = method parameter
  %  Outputs : y = approximated solution in the nodes

 time_istants=floor((T-t0)/h);
 t=linspace(t0, T, time_istants+1);
 y(:,1) = y0;
iter=[];
 for (n = 2:length(t))
   [y(:, n), ~,~, output] = fsolve( ...
      @(z) -z + y(:, n-1) + h* ( theta * f(t(n-1), y(:,n-1)) + (1-theta) * f(t(n), z) ) , ...
      y(:, n-1), ...
      optimset('Display', 'off', 'MaxIter', 1000) ...
   );
    iter=[iter output.iterations];
 end

end
%% Exercise 11.6
% Implement the Theta Method method in a suitable function
clc;clear; close all;
%%
f   = @(t,y) -2.*t.*y;
y_ex = @(t) exp(-t.^2);

theta = 0; %EE
%theta = 1/4;
theta = 1/2; % CN
%theta = 3/4;
%theta = 1; %EI
err=[];
for n = 4:8
 h = 2^(-n);


 [t,y] = theta_method(f, 1, 0, 1, h,theta);
 err= [err max(abs(y - y_ex(t)))];

end

p = log( err(1:end-1)./(err(2:end) )) / log(2)
% The order of convergence is 1 for theta ~= 1/2 and 2 for theta = 1/2, as expected.
```